
Query Optimization: Index, and Materialized Views

In the third Lab we will proceed with other notions and applications of *DB optimization*. Where we will see more concepts to enhance the performance of *relational databases*. The first concept is the *Index*, which represents a powerful mechanism for improving query efficiency by facilitating rapid data retrieval. The second one is *Materialized views*, which represents an object that *stores the results of a query* as a table. *Indexes, and Materialized views* are interrelated concepts in the sense that they all contribute to the optimization of database queries.

① Useful Notions & Syntaxes

(a) Index

In a database is a structure employed to enhance query performance by providing a faster way to retrieve data. It is an important component of database optimization, especially when dealing with large datasets and complex queries. The basic syntax is:

-----Syntax Index

```
CREATE INDEX index_name  
ON table_name (column1, column2, ...);
```

-----Drop Index

```
DROP INDEX [schema_name.]index_name;
```

There are different types of indexes, like:

B-Tree Index. The most common type of index, it was already presented at the beginning.

Bitmap Index. Is a way to optimize querying for columns with a small set of distinct values (i.e. Low cardinality, e.g. Gender in [*Male, Female*]). The use of bitmaps is efficient for representing and retrieving data, where certain columns have a limited number. The syntax is:

----- BITMAP INDEX Creation

```
CREATE BITMAP INDEX index_name  
ON table_name (column_name);
```

(b) Materialized views

- i) A materialized view is essentially a database *physical object* that contains the *results of a query* (i.e. store precomputed data);
- ii) It stores the data, which can be refreshed to reproduce changes in the primary data;

- iii) Materialized views can be used to improve query performance by precomputing and storing the results of *complex or computationally expensive queries* that involve *aggregations, joins,...*, where the execution time of such queries can be significant.
- iv) The syntax for creating a materialized view is as follows:

----- MATERIALIZED VIEW Creation

```
CREATE MATERIALIZED VIEW MV_viewName
ENABLE QUERY REWRITE--DB uses the MV instead of original tables
AS
SQL STATEMENT (SELECT)
```

- v) *Refreshing Materialized Views.* The refresh process for materialized views refers to the *update* of the data stored within it to reflect changes in the primary tables or data sources. This is crucial considering that the materialized views store the results of a query at a *specific point in time*, and over time, the data in the original tables may change. Therefore, to keep the materialized view *up-to-date*, is needed to refresh it;
- vi) The refresh process can be done *manually* or *scheduled to occur at specific intervals*:
 - *Manual Refresh.* This can be done at any time by executing a specific *SQL or PL/SQL command*, as follows:

```
-- Refreshing the materialized view
EXEC DBMS_MVIEW.REFRESH('MV_viewName');
```

- *Scheduled Refresh.* Instead of refreshing materialized views manually, you can set up a schedule to refresh them *automatically* at specific periods. In this case, *DBMS_SCHEDULER* or other scheduling mechanisms are called to schedule regular refreshes.
- vii) To disable a *query rewrite* for a materialized view, it can be done as follows:

```
---- Desiable

ALTER MATERIALIZED VIEW MV_viewName
DISABLE QUERY REWRITE;
```

- viii) To drop a materialized view, the following query should be introduced:

```
-----Drop

DROP MATERIALIZED VIEW [schema.]MV_viewName;

DROP MATERIALIZED VIEW MV_viewName;

DROP MATERIALIZED VIEW my_materialized_view CASCADE CONSTRAINTS;
```

ix) **Important**

- Materialized views consume storage space as they physically store the data.
- The refresh process may influence system performance, especially if the materialized view is large or the refresh frequency is high.
- The materialized view stores precomputed results, reducing the need for complex computations during query execution (i.e. instead of calculating aggregations or performing complex joins at runtime, the database can retrieve the results directly from the materialized view, which is a faster option).

2 Lab Work

- Where to create the best indexes in an SQL statement. Create Indexes for the queries : (i; ii; V; Vii; xii). What do you notice? Create the execution plan for these queries before and after the creation of the indexes. What do you notice?
- For the following queries, make the Execution plan (*Explain Plan*), *without "The Materialized Views"*, and *with "The Materialized Views"*. **Explain what do you notice.**

Queries

- Find the average rating for each show along with the total number of votes.
- Find shows where the average rating is higher than the overall average rating.
- Find staff members who have worked on shows with the highest average rating.*
- Find the top N staff members with the highest average rating across all shows. Where N is between 20 and 200.
- Find the staff member with the highest average rating for shows in a specific genre (Short, tvSeries, tvMovie).
- Find the staff member with the highest average rating for shows in each genre.
- Find shows that have more than one director.
- Find the staff member who has worked in the most diverse set of categories (e.g., director, actor, etc.).
- Find the top N episodes with the highest average rating and the most number of votes. Where N is between 5 and 10.
- Find shows that have consistently maintained an average rating above 6.5 across multiple seasons.
- Find the average rating trend for each episode in a show by comparing it with the average rating of the previous episode.
- Create a matrix showing the number of times each pair of staff members has collaborated in the same show.
- Find shows where the number of distinct staff members (across all categories) is above 5.
- Find the staff members who have been involved in the top 5 longest-running shows based on the difference between *startYear* and *endYear*.

3 Homework

The queries that you cannot finish at the lab session are considered homework. The report is going to be submitted later on. **You can work in groups of three maximum.**